

Lab0: Gitlab 实验报告

姓名：王思博

学号：25300190047

仓库地址：<https://github.com/sibo25/ICS-Lab0>

第一题：认真阅读文档，学习 Git 的基本用法，并在报告中回答文档中的问题。

- 你之前有过多人协同开发的经历吗？如果有，你们是使用什么方式分工协作的？

答：在本次实验之前，我没有多人协同开发的经历，主要是独立编写和修改代码。通过本次实验，我初步了解了如何使用 Git 创建分支、记录修改、合并代码及解决冲突，为今后的协作开发做准备。

- 思考一下，Git 为什么要设计“暂存-提交”两个步骤？

答：Git 将版本记录分为“暂存”和“提交”两个步骤，是为了将修改内容的选择与版本的正式保存分开。暂存区允许我们从工作区中选择与当前任务相关的修改，经过检查和整理后，再通过提交将其记录为一个版本。这样可以避免将不同任务或没有完全做好的修改混入同一次提交，使每次提交具有完整的逻辑，便于后问题定位和版本回退。

- `git branch` 和 `git branch -a` 的区别是什么？查阅资料并回答。

答：`git branch`：列出本地分支，例如 `main`、`feature`；`git branch -a`：列出本地分支，以及本地记录的远程跟踪分支，例如 `remotes/origin/main`。远程跟踪分支用于在本地记录远程分支的状态，通常显示为 `remotes/远程名称/分支名称`。两条命令都不会主动获取远程仓库的最新信息，如需更新这些记录，可先执行 `git fetch`。

```
● sibo_wang@sibo:~/ICS-Lab0$ git branch -a
  feature
* main
  remotes/origin/HEAD -> origin/main
  remotes/origin/main
○ sibo_wang@sibo:~/ICS-Lab0$
```

第二题、第四题：我首先在 Ubuntu 环境中配置了 Git 的用户名和邮箱，并生成 SSH 密钥，将公钥添加到 GitHub，完成连接验证。随后，我使用课程提供的模板创建了个人仓库 ICS-Lab0，并将其克隆到本地。通过 VSCode 打开该仓库后，我修改了 main.c 中的打印文字，依次使用 git add、git commit 和 git push 完成暂存、本地提交及上传到 GitHub。

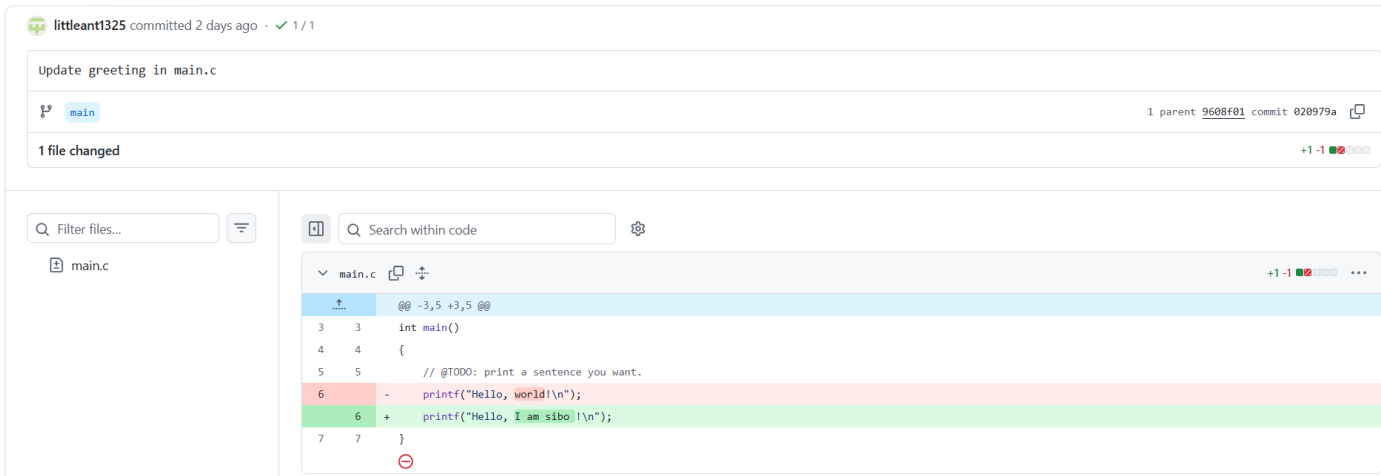
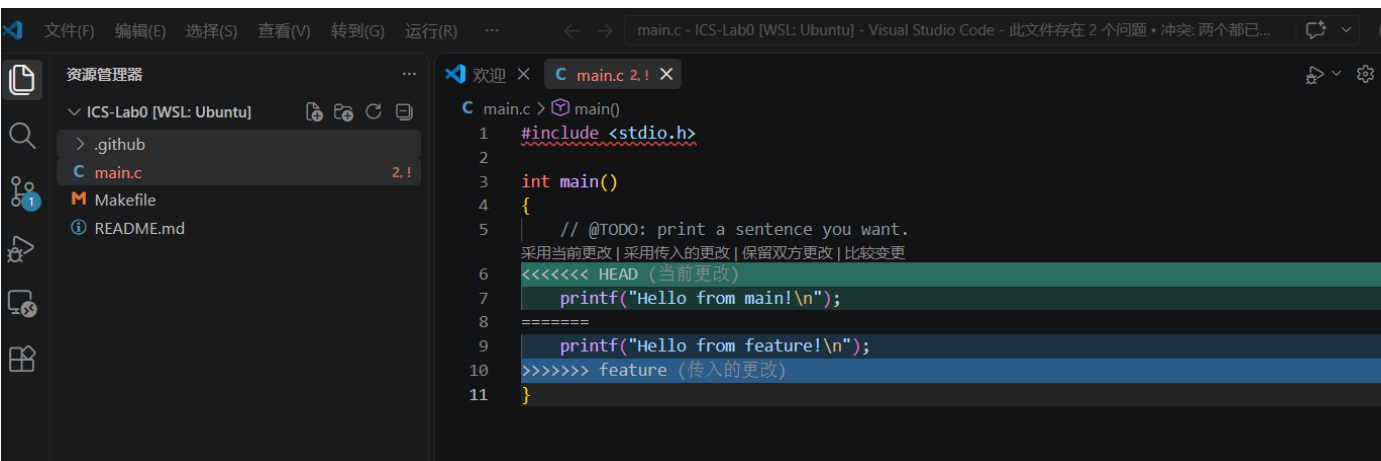


图 1：修改 main.c 的输出文字

我使用 git switch -c feature 创建并切换到 feature 分支，将 main.c 中的输出文字修改为“Hello from feature!”，暂存并提交。随后切换回 main 分支，将同一行修改为“Hello from main!”，再次暂存并提交。在 main 分支执行 git merge feature 时，由于两个分支对同一行进行了不同修改，Git 无法自动合并，提示发生内容冲突。合并冲突如图 2 所示：



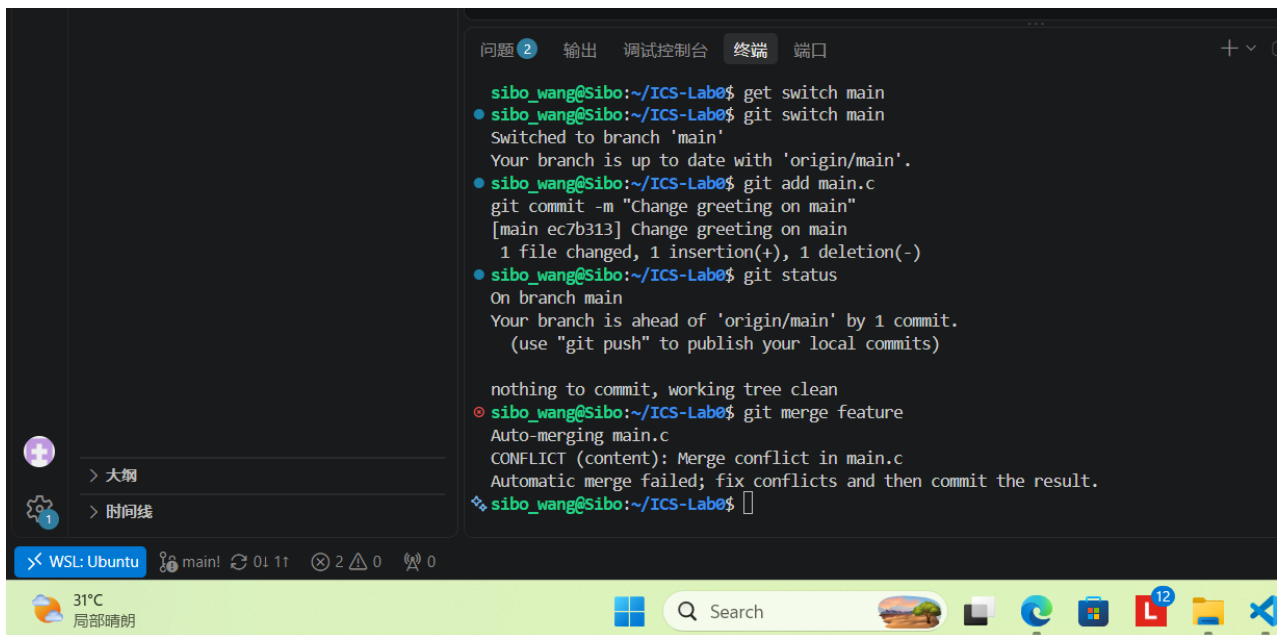
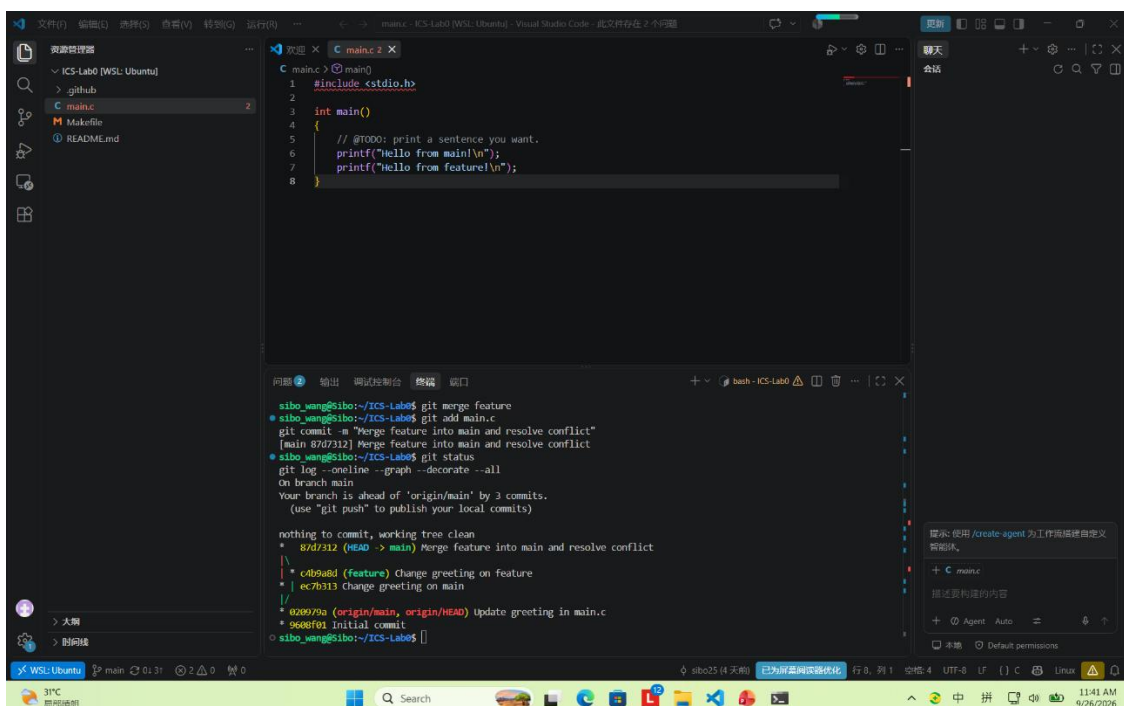


图 2

我在 VSCode 中选择“保留双方更改”，保留两条 printf 语句并保存文件，再执行 `git add main.c` 标记冲突已解决，通过 `git commit` 创建合并提交。最后检查仓库状态和提交历史，确认工作区干净、两个分支的历史已汇合，并将 main 分支推送至 GitHub。



第 3 题:

1. 第一篇:《Commit message 和 Change log 编写指南》

这篇文章介绍了规范编写提交说明和版本更新日志的方法。提交说明应清楚地表达本次修改的类型、范围和目的,必要时进一步解释修改原因及其影响。统一的格式有助于开发者阅读、检索和理解提交历史,也便于借助工具生成更新日志。其中,Commit message 描述单次提交,Change log 则汇总一个版本的主要变化。通过阅读,我认识到,版本管理不仅需要保存代码,还需要记录修改的意图。因此,每次提交应围绕明确的任务展开,并使用准确、具体的提交说明。

2. 第二篇:《Gitflow 使用规范》

这篇文章介绍了利用不同分支组织开发、测试和发布工作的 Gitflow 流程。其中, master 分支保存稳定的发布版本, develop 分支用于汇集开发成果, feature 分支用于开发独立功能, release 分支用于发布前的测试和修复, hotfix 分支用于紧急修复已发布版本的问题。通过明确各类分支的职责及合并关系,团队能够更有序地开展并行开发和版本维护。本次实验中,我创建了 feature 分支进行独立修改,并通过将其合并到 main 分支,练习了冲突解决与合并提交。结合本次实验,我理解了分支隔离修改、合并汇集成果的作用,同时认识到分支管理不能完全消除冲突,发生冲突时仍需开发者判断并解决。

3. 为什么我们要学习 Git

我认为我们学习 Git 是为了更可靠地管理代码变化,并为团队协作建立基础。Git 能够记录各次提交,使我们可以查看历史、比较修改,并在出现问题时查找原因或恢复先前的版本。分支机制允许开发者独立尝试新功能,再通过合并整合成果;远程仓库则便于共享代码和保存已推送的版本。

通过本次实验,我掌握了暂存、提交、分支、合并和推送等基本操作,也明确了本地提交与上传 GitHub 的区别。解决合并冲突的过程让我认识到,版本管理不仅是使用命令,还需要理解不同修改之间的关系。学习 Git 有助于形成清晰记录修改、按任务组织提交和主动检查合并结果的习惯,为后续课程实验与团队项目打下基础。